

1. Main.java

```
package miniproject;

import javax.swing.*;
import java.sql.*;
import java.util.HashMap;
import java.util.Map;
import java.awt.Dimension;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.RenderingHints;
import java.awt.Color;

public class Main {

    private static final String DB_URL =
"jdbc:mysql://localhost:3306/mini?useSSL=false&serverTimezone=UTC"
; private static final String DB_USER = "root";
    private static final String DB_PASS = "mansi1106";

    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            JOptionPane.showMessageDialog(null, "MySQL Driver not found!");
        }
        return;
    }

    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); }
    })
}
```

```
catch (Exception ignored) {}
```

```
new EshopGUI.WelcomeFrame();
```

```
});
```

```
}
```

```
public static Connection getConnection() throws SQLException {
```

```
return DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
```

```
}
```

```
static class Company_portal {
```

```
public static boolean companyLogin(Connection conn, String email, String password) throws  
SQLException {
```

```
String sql = "SELECT company_id, company_name, owner_name, category, state FROM  
company WHERE email = ? AND password = ?";
```

```
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
pstmt.setString(1, email);
```

```
pstmt.setString(2, password);
```

```
try (ResultSet rs = pstmt.executeQuery()) {
```

```
if (rs.next()) {
```

```
System.out.println("\nWelcome, " + rs.getString("company_name")); return
```

```
true;
```

```
}
```

```
return false;
```

```
}
```

```
}
```

```
}
```

```
// 1 PRODUCT ANALYTICS
```

```
public static void viewProductAnalytics(Connection conn, int companyId) throws SQLException {
```

```
String sql = "SELECT p.product_name, COUNT(v.view_id) AS total_views " +
```

```
"FROM product p LEFT JOIN view v ON p.product_id = v.product_id " +  
"WHERE p.company_id = ? " +  
"GROUP BY p.product_id, p.product_name " + "ORDER  
BY total_views DESC";
```

```
Map<String, Integer> dataset = new LinkedHashMap<>();
```

```
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {  
    pstmt.setInt(1, companyId);  
    try (ResultSet rs = pstmt.executeQuery()) {  
        while (rs.next()) {  
            String name = rs.getString("product_name"); int views  
            = rs.getInt("total_views");  
            dataset.put(name, views);  
        }  
    }  
}
```

```
if (dataset.size() > 0) {  
    SimpleBarChart chartPanel = new SimpleBarChart(dataset);  
    chartPanel.setPreferredSize(new Dimension(800, 600));
```

```
JFrame frame = new JFrame("Product Analytics - Views");  
frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
JScrollPane scrollPane = new JScrollPane(chartPanel);  
frame.add(scrollPane);
```

```
frame.setSize(800, 600);  
frame.setLocationRelativeTo(null);  
frame.setAlwaysOnTop(true);
```

```

frame.setVisible(true);
} else {

OptionPane.showMessageDialog(null, "No product views data available."); }

}

// 2 SALES ANALYTICS

public static void viewSalesAnalytics(Connection conn, int companyId) throws SQLException {

System.out.println("\n--- SALES ANALYTICS (from User Carts) ---");

String sql = "SELECT p.product_name, " +
"COALESCE(SUM(c.quantity), 0) as total_in_carts, " +
"COALESCE(SUM(c.quantity * p.price), 0) as total_revenue_in_carts " + "FROM
product p " +
"LEFT JOIN cart c ON p.product_id = c.product_id " +
"WHERE p.company_id = ? " +
"GROUP BY p.product_id, p.product_name " +
"ORDER BY total_revenue_in_carts DESC";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

pstmt.setInt(1, companyId);

ResultSet rs = pstmt.executeQuery();

System.out.println("Product Name\t\tUnits in Carts\tRevenue in Carts");

System.out.println("-----");

double totalCompanyRevenue = 0;

int totalUnitsSold = 0;

while (rs.next()) {

String productName = rs.getString("product_name");

```

```
int unitsSold = rs.getInt("total_in_carts");
double revenue = rs.getDouble("total_revenue_in_carts");
```

```
System.out.printf("%-20s\t%d\t\t₹%.2f\n", productName, unitsSold, revenue);
```

```
totalUnitsSold += unitsSold;
```

```
totalCompanyRevenue += revenue;
```

```
}
```

```
System.out.println("-----");
```

```
System.out.printf("TOTAL (in Carts):\t\t%d\t\t₹%.2f\n", totalUnitsSold,
totalCompanyRevenue);
```

```
} catch (SQLException e) {
```

```
System.out.println("Sales data not available yet.");
```

```
}
```

```
}
```

```
// MANAGE PRODUCTS
```

```
public static void addProduct(Connection conn, int companyId, String name, String category,
double price, String description, String imagePath) throws SQLException {
```

```
String sql = "INSERT INTO product (company_id, product_name, category, price, description,
image_path, rating) VALUES (?, ?, ?, ?, ?, ?, 0.0)";
```

```
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
pstmt.setInt(1, companyId);
```

```
pstmt.setString(2, name);
```

```
pstmt.setString(3, category);
```

```
pstmt.setDouble(4, price);
```

```
pstmt.setString(5, description);
```

```
pstmt.setString(6, imagePath);
```

```
int rows = pstmt.executeUpdate();
```

```

if (rows > 0) {
    System.out.println("Product added successfully!");

    addNotification(conn, companyId, "New product '" + name + "' added successfully!"); } else {

    System.out.println("Failed to add product.");
}
}
}

```

```

public static void updateProduct(Connection conn, int companyId, int productId, String
newName, String newCategory, double newPrice, String newDesc, String newImg)
throws SQLException {

```

```

    String sql = "UPDATE product SET product_name = ?, category = ?, price = ?, description = ?,
image_path = ? WHERE product_id = ? AND company_id = ?";

```

```

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

```

```

        pstmt.setString(1, newName);

```

```

        pstmt.setString(2, newCategory);

```

```

        pstmt.setDouble(3, newPrice);

```

```

        pstmt.setString(4, newDesc);

```

```

        pstmt.setString(5, newImg);

```

```

        pstmt.setInt(6, productId);

```

```

        pstmt.setInt(7, companyId);

```

```

        int rows = pstmt.executeUpdate();

```

```

        if (rows > 0) {

```

```

            System.out.println("Product updated successfully!");

```

```

            addNotification(conn, companyId, "Product '" + newName + "' updated successfully!"); } else {

```

```

            System.out.println("Update failed! Product not found or you don't have permission."); }

```

```

        }

```

```

    }

```

```

    public static void deleteProduct(Connection conn, int companyId, int productId) throws
SQLException {

```

```

        String productName = getProductName(conn, productId, companyId);

```

```

String sql = "DELETE FROM product WHERE product_id = ? AND company_id = ?"; try
(PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setInt(1, productId);
    pstmt.setInt(2, companyId);

    int rows = pstmt.executeUpdate();
    if (rows > 0) {
        System.out.println("Product deleted successfully!");
        if (productName != null) {
            addNotification(conn, companyId, "Product '" + productName + "' deleted!"); }
        } else {
            System.out.println("Product not found or you don't have permission!"); }
        }
    }

    // 4 VIEW NOTIFICATIONS

    public static void viewNotifications(Connection conn, int companyId) throws SQLException {
        System.out.println("\n--- COMPANY NOTIFICATIONS ---");

        String sql = "SELECT notification_no, message, date_sent FROM notification WHERE
        company_id = ? ORDER BY date_sent DESC LIMIT 10";

        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, companyId);

            ResultSet rs = pstmt.executeQuery();

            boolean hasNotifications = false;
            while (rs.next()) {
                hasNotifications = true;

                System.out.println("[" + rs.getTimestamp("date_sent") + "] " +
                rs.getString("message"));
            }
        }
    }

```

```

if (!hasNotifications) {
    System.out.println("No notifications found.");
}
}
}

// 5 COMPANY PROFILE

public static void viewCompanyProfile(Connection conn, int companyId) throws SQLException {
    String sql = "SELECT * FROM company WHERE company_id = ?";
    PreparedStatement pstmt = conn.prepareStatement(sql);
    pstmt.setInt(1, companyId);
    ResultSet rs = pstmt.executeQuery();

    if (rs.next()) {
        System.out.println("\n--- COMPANY PROFILE ---");
        System.out.println("ID: " + rs.getInt("company_id"));
        System.out.println("Name: " + rs.getString("company_name"));
        System.out.println("Owner: " + rs.getString("owner_name"));
        System.out.println("Email: " + rs.getString("email"));
        System.out.println("Phone: " + rs.getString("phone"));
        System.out.println("Category: " + rs.getString("category"));
        System.out.println("Address: " + rs.getString("street") + ", " +
            rs.getString("city") + " - " + rs.getString("postal_code"));
        System.out.println("State: " + rs.getString("state"));
        System.out.println("Verification Status: " + (rs.getBoolean("verified") ? "Verified " : "Pending
        "));
    }
}

// HELPER METHODS

```

```

public static int getCompanyId(Connection conn, String email) throws SQLException {
String sql = "SELECT company_id FROM company WHERE email = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);

pstmt.setString(1, email);

ResultSet rs = pstmt.executeQuery();

if (rs.next()) return rs.getInt(1);

return -1;
}

```

```

private static void addNotification(Connection conn, int companyId, String message) throws
SQLException {

String sql = "INSERT INTO notification (company_id, message, date_sent) VALUES (?, ?,
NOW())";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

pstmt.setInt(1, companyId);

pstmt.setString(2, message);

pstmt.executeUpdate();

}

}

```

```

private static String getProductName(Connection conn, int productId, int companyId) throws
SQLException {

String sql = "SELECT product_name FROM product WHERE product_id = ? AND company_id =
?";

PreparedStatement pstmt = conn.prepareStatement(sql);

pstmt.setInt(1, productId);

pstmt.setInt(2, companyId);
ResultSet rs = pstmt.executeQuery();

return rs.next() ? rs.getString("product_name") : null; }

}

```

```

static class SimpleBarChart extends JPanel {

```

```

private Map<String, Integer> data;

private int barWidth = 100;

private int barGap = 50;

private int padding = 40;

private int labelPadding = 40;


public SimpleBarChart(Map<String, Integer> data) { this.data = (data
!= null) ? data : new LinkedHashMap<>(); int width = (this.data.size() *
(barWidth + barGap)) + (2 * padding); int height = 500;

setPreferredSize(new Dimension(width, height));

setBackground(Color.WHITE);

}

```

```

@Override

protected void paintComponent(Graphics g) {

super.paintComponent(g);

if (data.isEmpty()) {

g.drawString("No data to display.", 10, 20);

return;

}

```

```

Graphics2D g2 = (Graphics2D) g;

g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);


int maxValue = 0;

for (Integer value : data.values()) {

if (value > maxValue) {

maxValue = value;

}

}

```

```
if (maxValue == 0) maxValue = 1;
```

```
int chartWidth = getWidth() - (2 * padding);
```

```
int chartHeight = getHeight() - (2 * padding) - labelPadding;
```

```
g2.setColor(Color.BLACK);
```

```
g2.drawLine(padding, padding, padding, padding + chartHeight);
```

```
g2.drawString(String.valueOf(maxValue), padding - 30, padding + 10);
```

```
g2.drawString("0", padding - 15, padding + chartHeight);
```

```
g2.drawString("Views", padding - 35, padding + chartHeight / 2);
```

```
g2.drawLine(padding, padding + chartHeight, padding + chartWidth, padding +  
chartHeight);
```

```
int x = padding + barGap;
```

```
g2.setColor(new Color(70, 130, 180));
```

```
for (Map.Entry<String, Integer> entry : data.entrySet()) {
```

```
String label = entry.getKey();
```

```
int value = entry.getValue();
```

```
int barHeight = (int) ((double) value / maxValue * chartHeight);
```

```
int y = padding + chartHeight - barHeight;
```

```
g2.fillRect(x, y, barWidth, barHeight);
```

```
g2.setColor(Color.BLACK);
```

```
String valueStr = String.valueOf(value);
```

```
FontMetrics fm = g2.getFontMetrics();
```

```

int valueWidth = fm.stringWidth(valueStr);

g2.drawString(valueStr, x + barWidth / 2 - valueWidth / 2, y - 5);


int labelWidth = fm.stringWidth(label);

String truncatedLabel = label;

if(labelWidth > barWidth) {

    truncatedLabel = label.substring(0, Math.min(label.length(), 15)) + "...";

    labelWidth = fm.stringWidth(truncatedLabel);

}

g2.drawString(truncatedLabel, x + barWidth / 2 - labelWidth / 2, padding + chartHeight +
labelPadding - 10);


g2.setColor(new Color(70, 130, 180));

x += (barWidth + barGap);

}

}

}

}

```

2. EshopGUI.java

```

package miniproject;


import javax.imageio.ImageIO;
import javax.swing.*.*;

import javax.swing.border.EmptyBorder;

import java.awt.*.*;

import java.awt.event.*;

import java.awt.image.BufferedImage;

import java.io.File;

```

```

import java.io.IOException;

import java.sql.*;

import java.sql.Date;

import java.text.SimpleDateFormat;

import java.util.InputMismatchException;

import java.util.Map;

import java.util.LinkedHashMap;


public class EshopGUI {

    // USER PORTAL CLASSES

    // WELCOME FRAME

    static class WelcomeFrame extends JFrame {

        public WelcomeFrame() {

            setTitle("E-Shop Portal");

            setSize(700, 450);

            setLocationRelativeTo(null);

            setDefaultCloseOperation(EXIT_ON_CLOSE);

            setLayout(new BorderLayout());

            JLabel bgLabel = new JLabel();

            try {

                BufferedImage bg = ImageIO.read(new File("images/bg_login.jpg"));

                bgLabel.setIcon(new ImageIcon(bg.getScaledInstance(700, 450,
                    Image.SCALE_SMOOTH)));

            } catch (IOException e) {

                bgLabel.setOpaque(true);
                bgLabel.setBackground(new Color(230, 230, 230));

            }

            bgLabel.setLayout(new GridBagLayout());

            JPanel card = new JPanel();

            card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));

```

```

card.setBorder(new EmptyBorder(30, 60, 30, 60));

card.setBackground(new Color(255, 255, 255, 230));

JLabel title = new JLabel("Welcome to E-Shop", SwingConstants.CENTER);
title.setFont(new Font("Poppins", Font.BOLD, 28));

title.setAlignmentX(Component.CENTER_ALIGNMENT);

title.setForeground(new Color(44, 62, 80));

card.add(title);

card.add(Box.createRigidArea(new Dimension(0, 25)));

JButton userBtn = new JButton(" Login as User");

JButton companyBtn = new JButton(" Login as Company");

styleMainButton(userBtn, new Color(52, 152, 219));

styleMainButton(companyBtn, new Color(46, 204, 113));

userBtn.addActionListener(e -> {

dispose();

new UserLoginFrame();

});

companyBtn.addActionListener(e -> {

dispose();

new CompanyPortalGUI();

});

card.add(userBtn);

card.add(Box.createRigidArea(new Dimension(0, 15)));

card.add(companyBtn);

bgLabel.add(card);

add(bgLabel, BorderLayout.CENTER);

setVisible(true);

}

```

```

private void styleMainButton(JButton btn, Color color) {
    btn.setFont(new Font("SansSerif", Font.BOLD, 18));
    btn.setBackground(color);
    btn.setForeground(Color.BLACK);
    btn.setFocusPainted(false);
    btn.setAlignmentX(Component.CENTER_ALIGNMENT);
    btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    btn.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));
}
}

// USER LOGIN FRAME
static class UserLoginFrame extends JFrame {
    private JTextField emailField;
    private JPasswordField passField;
    public UserLoginFrame() {
        setTitle("User Login - E-Shop");
        setSize(500, 400);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        JLabel bg = new JLabel();
        try {
            BufferedImage img = ImageIO.read(new File("images/user_bg.jpg")); bg.setIcon(new
            ImageIcon(img.getScaledInstance(500, 400, Image.SCALE_SMOOTH))); } catch (IOException
            e) {
                bg.setOpaque(true);
                bg.setBackground(new Color(240, 240, 240));
            }
        bg.setLayout(new GridBagLayout());
        JPanel card = new JPanel();

```

```
card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));

card.setBackground(new Color(255, 255, 255, 245));

card.setBorder(new EmptyBorder(25, 40, 25, 40));


JPanel titlePanel = new JPanel();

titlePanel.setOpaque(false);

titlePanel.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 5));

try {

    JLabel icon = new JLabel(new ImageIcon(ImageIO.read(new File("images/app_icon.jpeg"))
        .getScaledInstance(45, 45, Image.SCALE_SMOOTH)));

    titlePanel.add(icon);

} catch (IOException ignored) {}

JLabel title = new JLabel("User Login");

title.setFont(new Font("Poppins", Font.BOLD, 24));

title.setForeground(new Color(41, 128, 185));

titlePanel.add(title);

card.add(titlePanel);

card.add(Box.createRigidArea(new Dimension(0, 20)));

JLabel emailLbl = new JLabel("Email:");

emailLbl.setForeground(Color.BLACK);

card.add(emailLbl);

emailField = new JTextField();

emailField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 35));

card.add(emailField);

card.add(Box.createRigidArea(new Dimension(0, 10)));

JLabel passLbl = new JLabel("Password:");

passLbl.setForeground(Color.BLACK);

card.add(passLbl);

passField = new JPasswordField();

passField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 35));
```

```
card.add(passField);

card.add(Box.createRigidArea(new Dimension(0, 20)));

JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 0));
buttonPanel.setOpaque(false);

JButton backBtn = new JButton("Back");
backBtn.setFont(new Font("SansSerif", Font.BOLD, 16));
backBtn.setBackground(new Color(108, 122, 137));
backBtn.setForeground(Color.WHITE);
backBtn.setOpaque(true);
backBtn.setBorderPainted(false);
backBtn.setFocusPainted(false);
backBtn.addActionListener(e -> {
    dispose();
    new WelcomeFrame();
});

JButton loginBtn = new JButton("Login");
loginBtn.setFont(new Font("SansSerif", Font.BOLD, 16));
loginBtn.setBackground(new Color(41, 128, 185));
loginBtn.setForeground(Color.BLACK);
loginBtn.setFocusPainted(false);
loginBtn.addActionListener(e -> attemptLogin());

buttonPanel.add(backBtn);
buttonPanel.add(loginBtn);

card.add(buttonPanel);
card.add(Box.createRigidArea(new Dimension(0, 12)));
```

```
JPanel linksPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 0));
linksPanel.setOpaque(false);
```

```
JLabel forgot = new JLabel("<HTML><U>Forgot Password?</U></HTML>");
forgot.setForeground(new Color(52, 152, 219));

forgot.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
forgot.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        dispose();
        new ForgotPasswordFrame();
    }
});
linksPanel.add(forgot);
```

```
JLabel register = new JLabel("<HTML><U>Don't have an account? Register</U></HTML>");
register.setForeground(new Color(46, 204, 113));

register.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
register.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        dispose();
        new RegisterFrame();
    }
});
linksPanel.add(register);
```

```
card.add(linksPanel);
```

```
bg.add(card);
```

```
add(bg);
```

```

setVisible(true);
}

private void attemptLogin() {
String email = emailField.getText().trim();
String pass = new String(passField.getPassword());
if (email.isEmpty() || pass.isEmpty()) {
JOptionPane.showMessageDialog(this, "Enter both fields!");
return;
}

try (Connection conn = Main.getConnection()) {
PreparedStatement ps = conn.prepareStatement(
"SELECT user_id, name FROM user WHERE email=? AND password=?");
ps.setString(1, email);
ps.setString(2, pass);
ResultSet rs = ps.executeQuery();
if (rs.next()) {
dispose();
new UserDashboard(rs.getInt("user_id"), rs.getString("name")); } else {
JOptionPane.showMessageDialog(this, "Invalid credentials!"); }
} catch (SQLException ex) {
JOptionPane.showMessageDialog(this, "DB Error: " + ex.getMessage()); }
}
}

// FORGOT PASSWORD FRAME
static class ForgotPasswordFrame extends JFrame {
private JTextField phoneField;

private JPasswordField newPassField;
public ForgotPasswordFrame() {
setTitle("Reset Password");

```

```
setSize(420, 320);

setLocationRelativeTo(null);

setDefaultCloseOperation(EXIT_ON_CLOSE);

JPanel panel = new JPanel();

panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

panel.setBorder(new EmptyBorder(30, 60, 30, 60));

panel.setBackground(new Color(245, 246, 250));


JLabel title = new JLabel("Forgot Password");

title.setFont(new Font("Poppins", Font.BOLD, 20));

title.setAlignmentX(Component.CENTER_ALIGNMENT);

panel.add(title);

panel.add(Box.createRigidArea(new Dimension(0, 15)));

JLabel lblPhone = new JLabel("Enter Phone Number:");

lblPhone.setForeground(Color.BLACK);

panel.add(lblPhone);

phoneField = new JTextField();

phoneField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 35));

panel.add(phoneField);

panel.add(Box.createRigidArea(new Dimension(0, 10)));

JLabel lblPass = new JLabel("New Password:");

lblPass.setForeground(Color.BLACK);

panel.add(lblPass);

newPassField = new JPasswordField();

newPassField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 35));

panel.add(newPassField);

panel.add(Box.createRigidArea(new Dimension(0, 15)));

JButton resetBtn = new JButton("Update Password");

resetBtn.setFont(new Font("SansSerif", Font.BOLD, 14));

resetBtn.setBackground(new Color(230, 126, 34));
```

```

resetBtn.setForeground(Color.BLACK);

resetBtn.setFocusPainted(false);

resetBtn.setAlignmentX(Component.CENTER_ALIGNMENT);

resetBtn.addActionListener(e -> updatePassword());

panel.add(resetBtn);


panel.add(Box.createRigidArea(new Dimension(0, 10))); JLabel back =
new JLabel("<HTML><U>Back to Login</U></HTML>");
back.setForeground(new Color(52, 152, 219));

back.setAlignmentX(Component.CENTER_ALIGNMENT);

back.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

back.addMouseListener(new MouseAdapter() {

    public void mouseClicked(MouseEvent e) {

        dispose();

        new UserLoginFrame();

    }

});

panel.add(back);


add(panel);

setVisible(true);

}

private void updatePassword() {

    String phone = phoneField.getText().trim();

    String newPass = new String(newPassField.getPassword()); if

(phone.isEmpty() || newPass.isEmpty()) {

        JOptionPane.showMessageDialog(this, "Please fill both fields!");

    }

    return;

}

try (Connection conn = Main.getConnection()) {

```

```

PreparedStatement ps = conn.prepareStatement("SELECT * FROM user WHERE
contact_no=?");

ps.setString(1, phone);

ResultSet rs = ps.executeQuery();

if (rs.next()) {

    PreparedStatement upd = conn.prepareStatement("UPDATE user SET password=? WHERE
contact_no=?");

    upd.setString(1, newPass);

    upd.setString(2, phone);

    upd.executeUpdate();

    JOptionPane.showMessageDialog(this, "Password Updated!");

    dispose();

    new UserLoginFrame();

} else {

    JOptionPane.showMessageDialog(this, "Phone not found!"); }

} catch (SQLException ex) {

    JOptionPane.showMessageDialog(this, "DB Error: " + ex.getMessage()); }

}

}

// REGISTER FRAME

static class RegisterFrame extends JFrame {

    private JTextField nameField = new JTextField();

    private JTextField emailField = new JTextField();

    private JPasswordField passField = new JPasswordField();

    private JTextField cityField = new JTextField();

    private JTextField areaField = new JTextField();

    private JTextField dobField = new JTextField();

    private JTextField contactField = new JTextField();

    public RegisterFrame() {
        setTitle("User Registration");
    }
}

```

```
setSize(480, 600);

setLocationRelativeTo(null);

setDefaultCloseOperation(EXIT_ON_CLOSE);


JPanel panel = new JPanel();

panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));

panel.setBorder(new EmptyBorder(20, 40, 20, 40));

panel.setBackground(new Color(252, 252, 252));


JLabel title = new JLabel("Create Account");

title.setFont(new Font("Poppins", Font.BOLD, 24));

title.setAlignmentX(Component.CENTER_ALIGNMENT);

panel.add(title);

panel.add(Box.createRigidArea(new Dimension(0, 20)));


panel.add(new JLabel("Full Name:"));

panel.add(nameField);

panel.add(Box.createRigidArea(new Dimension(0, 10)));

panel.add(new JLabel("Email:"));

panel.add(emailField);

panel.add(Box.createRigidArea(new Dimension(0, 10)));

panel.add(new JLabel("Password:"));

panel.add(passField);

panel.add(Box.createRigidArea(new Dimension(0, 10)));

panel.add(new JLabel("City:"));

panel.add(cityField);

panel.add(Box.createRigidArea(new Dimension(0, 10)));

panel.add(new JLabel("Area/Street:"));

panel.add(areaField);

panel.add(Box.createRigidArea(new Dimension(0, 10)));
```

```
panel.add(new JLabel("Date of Birth (YYYY-MM-DD):"));
panel.add(dobField);
panel.add(Box.createRigidArea(new Dimension(0, 10)));
panel.add(new JLabel("Contact Number:"));
panel.add(contactField);
panel.add(Box.createRigidArea(new Dimension(0, 20)));
```

```
Component[] components = panel.getComponents();
for (Component c : components) {
    if (c instanceof JTextField) {
        ((JTextField)c).setMaximumSize(new Dimension(Integer.MAX_VALUE, 35)); }
}
```

```
JButton registerBtn = new JButton("Register");
registerBtn.setFont(new Font("SansSerif", Font.BOLD, 16));
registerBtn.setBackground(new Color(46, 204, 113));
registerBtn.setForeground(Color.BLACK);
registerBtn.setFocusPainted(false);
registerBtn.setAlignmentX(Component.CENTER_ALIGNMENT);
registerBtn.addActionListener(e -> attemptRegistration());
panel.add(registerBtn);
```

```
panel.add(Box.createRigidArea(new Dimension(0, 10)));
JLabel back = new JLabel("<HTML><U>Already have an account? Login</U></HTML>");
back.setForeground(new Color(52, 152, 219));
back.setAlignmentX(Component.CENTER_ALIGNMENT);
back.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
back.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        dispose();
        new UserLoginFrame();
    }
});
```

```

    }
    });

    panel.add(back);

    add(new JScrollPane(panel));
    setVisible(true);
}

private void attemptRegistration() {
    String name = nameField.getText().trim();
    String email = emailField.getText().trim();
    String pass = new String(passField.getPassword());
    String city = cityField.getText().trim();
    String area = areaField.getText().trim();
    String dob = dobField.getText().trim();
    String contact = contactField.getText().trim();

    if (name.isEmpty() || email.isEmpty() || pass.isEmpty() || city.isEmpty() || area.isEmpty() ||
        dob.isEmpty() || contact.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please fill in all fields!");
    }
    return;
}

Date sqlDob;
long contactNum;

try {
    sqlDob = Date.valueOf(dob);
} catch (IllegalArgumentException e) {
    JOptionPane.showMessageDialog(this, "Invalid Date format. Use YYYY-MM-DD.");
    return;
}

```

```
}
```

```
try {
```

```
    contactNum = Long.parseLong(contact);
```

```
} catch (NumberFormatException e) {
```

```
    JOptionPane.showMessageDialog(this, "Invalid Contact Number. Use digits only.");
```

```
return;
```

```
}
```

```
String sql = "INSERT INTO user (name, email, password, city, area, dob, contact_no) VALUES (?,  
?, ?, ?, ?, ?, ?)";
```

```
try (Connection conn = Main.getConnection(); PreparedStatement ps =  
conn.prepareStatement(sql)) {
```

```
    ps.setString(1, name);
```

```
    ps.setString(2, email);
```

```
    ps.setString(3, pass);
```

```
    ps.setString(4, city);
```

```
    ps.setString(5, area);
```

```
    ps.setDate(6, sqlDob);
```

```
    ps.setLong(7, contactNum);
```

```
    ps.executeUpdate();
```

```
    JOptionPane.showMessageDialog(this, "Registration Successful! Please login.");
```

```
dispose();
```

```
new UserLoginFrame();
```

```
} catch (SQLException ex) {
```

```
    if (ex.getMessage().contains("Duplicate entry")) {
```

```
        JOptionPane.showMessageDialog(this, "This email is already registered!"); } else {
```

```

OptionPane.showMessageDialog(this, "Database Error: " + ex.getMessage()); }

}

}

}

// USER DASHBOARD

static class UserDashboard extends JFrame {

    private final int userId;

    private final String userName;

    private final Color COLOR_PRIMARY_BG = new Color(240, 240, 240);

    private final Color COLOR_ACCENT = new Color(5, 108, 137); // blue

    private final Color COLOR_HEADER_FG = Color.WHITE;

    private final Color COLOR_TEXT_DIM = new Color(120, 120, 120);

    private final Color COLOR_JASMINE_WHITE = new Color(0xF8F8F0);

    private final JComboBox<String> categoryFilter = new JComboBox<>();

    private final JTextField txtQuery = new JTextField();

    private final DefaultListModel<ProductModel> productListModel = new DefaultListModel<>();

    private final JList<ProductModel> productList = new JList<>(productListModel);

    private final JButton btnCart = flatNavButton(" Cart");

    public UserDashboard(int userId, String userName) {

        this.userId = userId;

        this.userName = userName;

        setTitle("E-Shop — Welcome, " + userName);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setSize(1180, 720);

        setLocationRelativeTo(null);

        JPanel content = new JPanel(new BorderLayout());
        content.setBackground(COLOR_PRIMARY_BG);

        setContentPane(content);
    }
}

```

```
JPanel topWrapper = new JPanel(new BorderLayout());
topWrapper.setBackground(COLOR_ACCENT);

JLabel banner = new JLabel();
try {
    BufferedImage b = ImageIO.read(new File("images/banner.jpg"));
    banner.setIcon(new ImageIcon(b.getScaledInstance(1180, 120, Image.SCALE_SMOOTH))); }
catch (Exception ex) {

    banner.setOpaque(true);
    banner.setBackground(COLOR_ACCENT);
    banner.setText(" ShopEase — curated picks for you");
    banner.setForeground(COLOR_HEADER_FG);
    banner.setFont(new Font("Poppins", Font.BOLD, 22));
    banner.setHorizontalAlignment(SwingConstants.LEFT);
    banner.setBorder(new EmptyBorder(20, 20, 20, 20));
}

topWrapper.add(banner, BorderLayout.NORTH);

JPanel navbar = new JPanel(new BorderLayout(12, 12));
navbar.setBackground(COLOR_ACCENT);
navbar.setBorder(new EmptyBorder(8, 16, 12, 16));

JLabel lblApp = new JLabel("Go Exploring");
lblApp.setFont(new Font("SansSerif", Font.BOLD, 24));
lblApp.setForeground(COLOR_HEADER_FG);
navbar.add(lblApp, BorderLayout.WEST);

JPanel searchPanel = new JPanel(new BorderLayout(8, 0));
searchPanel.setOpaque(false);
```

```
categoryFilter.setPreferredSize(new Dimension(170, 34));
categoryFilter.setFont(new Font("SansSerif", Font.PLAIN, 13));
searchPanel.add(categoryFilter, BorderLayout.WEST);
```

```
txtQuery.setFont(new Font("SansSerif", Font.PLAIN, 14));
txtQuery.setPreferredSize(new Dimension(420, 34));
searchPanel.add(txtQuery, BorderLayout.CENTER);
```

```
String placeholder = "Find a treasure...";
Color ghostColor = Color.GRAY;
Color activeColor = Color.BLACK;
txtQuery.setText(placeholder);
txtQuery.setForeground(ghostColor);
txtQuery.addFocusListener(new FocusAdapter() {
    @Override
    public void focusGained(FocusEvent e) {
        if (txtQuery.getText().equals(placeholder)) {
            txtQuery.setText("");
            txtQuery.setForeground(activeColor);
        }
    }
    @Override
    public void focusLost(FocusEvent e) {
        if (txtQuery.getText().isEmpty()) {
            txtQuery.setForeground(ghostColor);
            txtQuery.setText(placeholder);
        }
    }
});
```

```

JButton btnSearch = new JButton("Search");

btnSearch.setBackground(new Color(245, 190, 66));

btnSearch.setForeground(Color.BLACK);

btnSearch.setFocusPainted(false);

btnSearch.setBorder(BorderFactory.createEmptyBorder(6, 12, 6, 12));

searchPanel.add(btnSearch, BorderLayout.EAST);


navbar.add(searchPanel, BorderLayout.CENTER);


JPanel navRight = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 6));
navRight.setOpaque(false);


JButton btnHome = flatNavButton("Home");
JButton btnWishlist = flatNavButton("Wishlist");


JButton btnLogout = flatNavButton("Logout");


JLabel lblGreeting = new JLabel("Hey " + userName + ", happy shopping! ");
lblGreeting.setForeground(COLOR_HEADER_FG);
lblGreeting.setFont(new Font("SansSerif", Font.ITALIC, 13));


navRight.add(btnHome);
navRight.add(btnWishlist);
navRight.add(btnCart);
navRight.add(lblGreeting);
navRight.add(btnLogout);


navbar.add(navRight, BorderLayout.EAST);


topWrapper.add(navbar, BorderLayout.SOUTH);
content.add(topWrapper, BorderLayout.NORTH);
```

```
JPanel historyPanel = new JPanel(new BorderLayout());
historyPanel.setBackground(COLOR_PRIMARY_BG);

historyPanel.setBorder(BorderFactory.createCompoundBorder(
new EmptyBorder(12, 12, 12, 6),
BorderFactory.createTitledBorder("Recent Searches" ) ));

historyPanel.setPreferredSize(new Dimension(240, 0));
DefaultListModel<String> historyModel = new DefaultListModel<>();
JList<String> historyList = new JList<>(historyModel);
historyPanel.add(new JScrollPane(historyList), BorderLayout.CENTER);
content.add(historyPanel, BorderLayout.WEST);


productList.setCellRenderer(new ProductRenderer());
productList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);


JScrollPane centerScroll = new JScrollPane(productList);
centerScroll.setBorder(new EmptyBorder(12, 12, 12, 12));
content.add(centerScroll, BorderLayout.CENTER);


JPanel bottom = new JPanel(new BorderLayout());
bottom.setBackground(COLOR_PRIMARY_BG);
bottom.setBorder(new EmptyBorder(8, 12, 12, 12));


JLabel status = new JLabel("Showing products");
status.setForeground(COLOR_TEXT_DIM);
bottom.add(status, BorderLayout.WEST);


JPanel quick = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
quick.setOpaque(false);

JButton btnRefresh = new JButton("Refresh");
```

```
btnRefresh.setBackground(COLOR_ACCENT);  
btnRefresh.setForeground(COLOR_HEADER_FG);
```

```
btnRefresh.setOpaque(true);  
btnRefresh.setBorderPainted(false);  
btnRefresh.setFocusPainted(false);  
quick.add(btnRefresh);  
bottom.add(quick, BorderLayout.EAST);
```

```
content.add(bottom, BorderLayout.SOUTH);
```

```
btnSearch.addActionListener(e -> performSearchAndSave(historyModel));  
txtQuery.addActionListener(e -> performSearchAndSave(historyModel));  
btnRefresh.addActionListener(e -> loadAllProducts());  
btnHome.addActionListener(e -> {  
    txtQuery.setText("");  
    categoryFilter.setSelectedIndex(0);  
    loadAllProducts();  
});  
btnWishlist.addActionListener(e -> openWishlistDialog());  
btnCart.addActionListener(e -> openCartDialog());  
btnLogout.addActionListener(e -> {  
    dispose();  
    new UserLoginFrame();  
});
```

```
historyList.addMouseListener(new MouseAdapter() {  
    public void mouseClicked(MouseEvent e) {  
        if (e.getClickCount() == 1) {  
            String kw = historyList.getSelectedValue();
```

```
if (kw != null) {  
    txtQuery.setText(kw);  
    performSearchAndSave(historyModel); }  
}  
}  
});
```

```
productList.addMouseListener(new MouseAdapter() {  
    public void mouseClicked(MouseEvent e) { if  
    (e.getClickCount() == 2) {  
        ProductModel p = productList.getSelectedValue(); if (p !=  
        null) openProductDetailDialog(p); }  
    }  
});
```

```
loadCategories();  
loadSearchHistory(historyModel);  
loadAllProducts();  
updateCartButtonCount();  
setVisible(true);  
}
```

```
private JButton flatNavButton(String text) {  
    JButton b = new JButton(text);  
    b.setFont(new Font("SansSerif", Font.BOLD, 13));  
    b.setForeground(COLOR_HEADER_FG);  
    b.setOpaque(false);  
    b.setContentAreaFilled(false);  
    b.setBorderPainted(false);  
    b.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));  
    b.addMouseListener(new MouseAdapter() {
```

```
public void mouseEntered(MouseEvent e) { b.setForeground(new Color(200, 235, 255)); }  
public void mouseExited(MouseEvent e) { b.setForeground(COLOR_HEADER_FG); } };  
  
return b;  
}
```

```
private void updateCartButtonCount() {  
    int itemCount = getCartItemCount(this.userId);  
    btnCart.setText(String.format(" Cart (%d)", itemCount));  
}
```

```
private int getCartItemCount(int userId) {  
    String sql = "SELECT get_cart_item_count(?)";  
    try (Connection c = Main.getConnection();  
        PreparedStatement ps = c.prepareStatement(sql)) {  
  
        ps.setInt(1, userId);  
        try (ResultSet rs = ps.executeQuery()) {  
            if (rs.next()) {  
                return rs.getInt(1);  
            }  
        }  
    } catch (SQLException e) {  
        System.err.println("Error getting cart count: " + e.getMessage());  
    }  
    return 0;  
}
```

```
private void loadCategories() {  
    categoryFilter.removeAllItems();  
    categoryFilter.addItem("All Categories");  
}
```

```
String sql = "SELECT DISTINCT category FROM product WHERE category IS NOT NULL ORDER BY category";
```

```
try (Connection c = Main.getConnection(); Statement st = c.createStatement(); ResultSet rs = st.executeQuery(sql)) {
```

```
while (rs.next()) categoryFilter.addItem(rs.getString(1));
```

```
} catch (SQLException ex) { }
```

```
}
```

```
private void loadSearchHistory(DefaultListModel<String> historyModel) {
```

```
historyModel.clear();
```

```
String sql = "SELECT DISTINCT search_keyword FROM search_history WHERE user_id=? AND search_keyword IS NOT NULL AND search_keyword<>\" ORDER BY search_date DESC LIMIT 20";
```

```
try (Connection c = Main.getConnection(); PreparedStatement ps = c.prepareStatement(sql)) {
```

```
ps.setInt(1, userId);
```

```
try (ResultSet rs = ps.executeQuery()) {
```

```
while (rs.next()) historyModel.addElement(rs.getString(1));
```

```
}
```

```
} catch (SQLException ex) { }
```

```
}
```

```
private void loadAllProducts() {
```

```
productListModel.clear();
```

```
String sql = "SELECT p.product_id, p.company_id, p.product_name, p.category, p.price, p.rating, p.image_path, c.company_name, p.description " +
```

```
"FROM product p LEFT JOIN company c ON p.company_id = c.company_id ORDER BY p.product_name LIMIT 200";
```

```
try (Connection c = Main.getConnection(); PreparedStatement ps = c.prepareStatement(sql); ResultSet rs = ps.executeQuery()) {
```

```
while (rs.next()) {
```

```
ProductModel p = new ProductModel();
```

```
p.id = rs.getInt("product_id");
```

```
p.companyId = rs.getInt("company_id");
```

```

p.name = rs.getString("product_name");
p.category = rs.getString("category");
p.price = rs.getDouble("price");
p.rating = rs.getDouble("rating");
p.imagePath = rs.getString("image_path");
p.companyName = rs.getString("company_name");
p.description = rs.getString("description");
productListModel.addElement(p);
}
} catch (SQLException ex) {
JOptionPane.showMessageDialog(this, "Error loading products: " + ex.getMessage()); }
}

```

```

private void performSearchAndSave(DefaultListModel<String> historyModel) {
String q = txtQuery.getText().trim();
if (q.equals("Find a treasure...")) {
q = "";
}

String category = (String) categoryFilter.getSelectedItem();
boolean hasCat = category != null && !category.equals("All Categories");

StringBuilder sb = new StringBuilder();

sb.append("SELECT p.product_id, p.company_id, p.product_name, p.category, p.price,
p.rating, p.image_path, c.company_name, p.description ")

.append("FROM product p LEFT JOIN company c ON p.company_id=c.company_id WHERE 1=1
");

if (!q.isEmpty()) {
sb.append(" AND (p.product_name LIKE ? OR p.category LIKE ? OR p.description LIKE ?) ");
}

if (hasCat) {
sb.append(" AND p.category = ? ");
}

```

```

}

sb.append(" ORDER BY p.product_name LIMIT 200");

int searchHistoryId = -1;

try (Connection c = Main.getConnection()) {
    PreparedStatement ps = c.prepareStatement(sb.toString()); int
idx = 1;

    if (!q.isEmpty()) {
        ps.setString(idx++, "%" + q + "%");
        ps.setString(idx++, "%" + q + "%");
        ps.setString(idx++, "%" + q + "%");
        searchHistoryId = saveSearchHistory(c, q);
        if (searchHistoryId > 0 && !historyModel.contains(q)) {
            historyModel.insertElementAt(q, 0);
            if (historyModel.getSize() > 20) historyModel.removeElementAt(20); }
        }

        if (hasCat) ps.setString(idx++, category);

        try (ResultSet rs = ps.executeQuery()) {
            productListModel.clear();
            while (rs.next()) {
                ProductModel p = new ProductModel();
                p.id = rs.getInt("product_id");
                p.companyId = rs.getInt("company_id");
                p.name = rs.getString("product_name");
                p.category = rs.getString("category");
                p.price = rs.getDouble("price");
                p.rating = rs.getDouble("rating");
                p.imagePath = rs.getString("image_path");
                p.companyName = rs.getString("company_name");
            }
        }
    }
}

```

```

p.description = rs.getString("description");
productListModel.addElement(p);

if (searchHistoryId > 0) {
    try {
        saveSearchResult(c, searchHistoryId, p.id);
    } catch (Exception ignore) {}
}
}
}
} catch (SQLException ex) {
    JOptionPane.showMessageDialog(this, "Search error: " + ex.getMessage());
}
}

```

```

private int saveSearchHistory(Connection c, String keyword) {

    String sql = "INSERT INTO search_history (user_id, search_keyword, query_text,
search_date) VALUES (?, ?, ?, NOW())";

    try (PreparedStatement ps = c.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

        ps.setInt(1, userId);

        ps.setString(2, keyword);

        ps.setString(3, keyword);

        ps.executeUpdate();

        try (ResultSet g = ps.getGeneratedKeys()) {
            if (g.next()) return g.getInt(1);
        }

    } catch (SQLException ex) {}

    return -1;
}

```

```

private void saveSearchResult(Connection c, int historyId, int productId) {

```

```

if (historyId <= 0) return;

String sql = "INSERT INTO search_result (search_history_id, product_id, relevance_score)
VALUES (?, ?, 1.0)";

try (PreparedStatement ps = c.prepareStatement(sql)) {
    ps.setInt(1, historyId);
    ps.setInt(2, productId);
    ps.executeUpdate();
} catch (SQLException ex) { }
}

// LOGIC TO ADD A PRODUCT VIEW

private void logProductView(int productId, int companyId) {
    new Thread(() -> {

        String sql = "INSERT INTO view (user_id, product_id, company_id, view_date) VALUES (?, ?, ?,
CURDATE())";

        try (Connection conn = Main.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setInt(1, userId);
            ps.setInt(2, productId);
            ps.setInt(3, companyId);
            ps.executeUpdate();
        } catch (SQLException e) {

            System.err.println("Could not log product view: " + e.getMessage()); }

    }).start();
}

private void openProductDetailDialog(ProductModel p) {
    logProductView(p.id, p.companyId);

    JDialog d = new JDialog(this, "Product — " + p.name, true);
    d.setSize(800, 900);

```

```
d.setLocationRelativeTo(this);  
d.getContentPane().setBackground(COLOR_PRIMARY_BG);
```

```
JPanel main = new JPanel();  
main.setLayout(new BoxLayout(main, BoxLayout.Y_AXIS));  
main.setBorder(new EmptyBorder(12, 12, 12, 12));  
main.setBackground(COLOR_PRIMARY_BG);
```

```
JPanel topDetailPanel = new JPanel(new BorderLayout());  
topDetailPanel.setBackground(COLOR_PRIMARY_BG);
```

```
JPanel left = new JPanel(new BorderLayout());  
left.setBackground(COLOR_PRIMARY_BG);  
left.setPreferredSize(new Dimension(320, 380)); JLabel img = new  
JLabel(loadImageIcon(p.imagePath, 300, 300));  
img.setHorizontalAlignment(SwingConstants.CENTER); left.add(img,  
BorderLayout.NORTH);  
topDetailPanel.add(left, BorderLayout.WEST);
```

```
JPanel right = new JPanel();  
right.setLayout(new BoxLayout(right, BoxLayout.Y_AXIS));  
right.setBackground(COLOR_PRIMARY_BG);  
right.setBorder(new EmptyBorder(6, 12, 6, 6));
```

```
JLabel lblName = new JLabel(p.name);  
lblName.setFont(new Font("SansSerif", Font.BOLD, 22));  
lblName.setForeground(COLOR_ACCENT);  
right.add(lblName);  
right.add(Box.createRigidArea(new Dimension(0, 10)));
```

```
JLabel lblCompany = new JLabel("Company: " + (p.companyName == null ? "-" :
```

```
p.companyName));  
  
lblCompany.setFont(new Font("SansSerif", Font.PLAIN, 14));  
right.add(lblCompany);  
  
JLabel lblCat = new JLabel("Category: " + (p.category == null ? "-" : p.category));  
lblCat.setFont(new Font("SansSerif", Font.PLAIN, 14));  
right.add(lblCat);  
  
JLabel lblPrice = new JLabel(String.format("Price: ₹ %.2f", p.price));  
lblPrice.setFont(new Font("SansSerif", Font.BOLD, 16));  
lblPrice.setForeground(new Color(39, 174, 96));  
right.add(lblPrice);  
  
JLabel lblRating = new JLabel(String.format("Rating: %.1f ★", p.rating));  
lblRating.setFont(new Font("SansSerif", Font.BOLD, 15));  
lblRating.setForeground(new Color(245, 190, 66));  
right.add(lblRating);  
right.add(Box.createRigidArea(new Dimension(0, 10)));  
  
JTextArea ta = new JTextArea();  
ta.setText(p.description == null ? "-" : p.description);  
ta.setLineWrap(true); ta.setWrapStyleWord(true);  
ta.setEditable(false);  
ta.setBackground(COLOR_JASMINE_WHITE);  
ta.setBorder(BorderFactory.createTitledBorder("Details"));  
ta.setMaximumSize(new Dimension(380, 140));  
right.add(ta);  
  
right.add(Box.createRigidArea(new Dimension(0, 12)));
```

```
JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT));  
actions.setOpaque(false);
```

```
JButton btnWish = new JButton(" Add to Wishlist");
```

```
JButton btnCart = new JButton(" Add to Cart");
```

```
JButton btnBuy = new JButton(" Buy Now");
```

```
btnWish.setBackground(COLOR_ACCENT); btnWish.setForeground(COLOR_HEADER_FG);  
btnWish.setOpaque(true); btnWish.setBorderPainted(false);
```

```
btnCart.setBackground(new Color(41, 128, 185));  
btnCart.setForeground(COLOR_HEADER_FG); btnCart.setOpaque(true);  
btnCart.setBorderPainted(false);
```

```
btnBuy.setBackground(new Color(230, 126, 34));  
btnBuy.setForeground(COLOR_HEADER_FG); btnBuy.setOpaque(true);  
btnBuy.setBorderPainted(false);
```

```
actions.add(btnWish);
```

```
actions.add(btnCart);
```

```
actions.add(btnBuy);
```

```
right.add(actions);
```

```
topDetailPanel.add(right, BorderLayout.CENTER);
```

```
topDetailPanel.setMaximumSize(new Dimension(Integer.MAX_VALUE, 450));
```

```
main.add(topDetailPanel);
```

```
JPanel feedbackPanelWrapper = new JPanel(new BorderLayout(10, 10));
```

```
feedbackPanelWrapper.setBackground(COLOR_PRIMARY_BG);
```

```
feedbackPanelWrapper.setBorder(BorderFactory.createTitledBorder("Customer Reviews"));
```

```
JPanel addFeedbackPanel = new JPanel();
```

```
addFeedbackPanel.setLayout(new BoxLayout(addFeedbackPanel, BoxLayout.Y_AXIS));
```

```
addFeedbackPanel.setOpaque(false);

addFeedbackPanel.setBorder(new EmptyBorder(5, 5, 10, 5));

JLabel lblLeaveReview = new JLabel("Leave a Review");
lblLeaveReview.setFont(new Font("SansSerif", Font.BOLD, 14));
addFeedbackPanel.add(lblLeaveReview);

JPanel ratingInputPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
ratingInputPanel.setOpaque(false);

ratingInputPanel.add(new JLabel("Your Rating:"));

String[] ratings = {"5 ★", "4 ★", "3 ★", "2 ★", "1 ★"};
JComboBox<String> ratingBox = new JComboBox<>(ratings);
ratingInputPanel.add(ratingBox);
addFeedbackPanel.add(ratingInputPanel);

JTextArea commentArea = new JTextArea(3, 1);
commentArea.setLineWrap(true);
commentArea.setWrapStyleWord(true);
JScrollPane commentScroll = new JScrollPane(commentArea);
addFeedbackPanel.add(commentScroll);

JButton submitFeedbackBtn = new JButton("Submit Feedback");
submitFeedbackBtn.setBackground(new Color(46, 204, 113));
submitFeedbackBtn.setForeground(Color.BLACK);
submitFeedbackBtn.setOpaque(true);
submitFeedbackBtn.setBorderPainted(false);
addFeedbackPanel.add(Box.createRigidArea(new Dimension(0, 5)));
addFeedbackPanel.add(submitFeedbackBtn);
feedbackPanelWrapper.add(addFeedbackPanel, BorderLayout.NORTH);
```

```

JPanel existingFeedbackPanel = new JPanel();

existingFeedbackPanel.setLayout(new BorderLayout(existingFeedbackPanel,
BoxLayout.Y_AXIS));

existingFeedbackPanel.setBackground(Color.WHITE);


JScrollPane feedbackScroll = new JScrollPane(existingFeedbackPanel);
feedbackScroll.setPreferredSize(new Dimension(100, 200));
feedbackPanelWrapper.add(feedbackScroll, BorderLayout.CENTER);
main.add(feedbackPanelWrapper);


JPanel relatedPanelWrapper = new JPanel(new BorderLayout());
relatedPanelWrapper.setBackground(COLOR_PRIMARY_BG);
relatedPanelWrapper.setBorder(BorderFactory.createTitledBorder("Related Products in " +
p.category));

JPanel relatedItemsPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 10));
JScrollPane relatedScroll = new JScrollPane(relatedItemsPanel);

relatedScroll.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_AS_NEEDED)
; relatedScroll.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_NEVER);
relatedScroll.setBorder(null);

relatedScroll.setPreferredSize(new Dimension(100, 150));
relatedPanelWrapper.add(relatedScroll, BorderLayout.CENTER);
main.add(relatedPanelWrapper);


d.add(main);


btnWish.addActionListener(e-> {

try (Connection c = Main.getConnection()) {
String ins = "INSERT INTO wishlist (user_id, product_id, added_at) VALUES (?, ?,
NOW())";

try (PreparedStatement ps = c.prepareStatement(ins)) {

```

```

ps.setInt(1, userId);
ps.setInt(2, p.id);
ps.executeUpdate();
JOptionPane.showMessageDialog(d, "Added to wishlist."); }
} catch (SQLException ex) {
JOptionPane.showMessageDialog(d, "Wishlist error: " + ex.getMessage()); }
});

```

```

btnCart.addActionListener(e -> {
String qtyStr = JOptionPane.showInputDialog(d, "Quantity:", "1"); if
(qtyStr == null) return;

```

```

int qty;
try {
qty = Integer.parseInt(qtyStr);
if (qty <= 0) {
JOptionPane.showMessageDialog(d, "Invalid quantity."); return;
}
} catch (NumberFormatException ex) {
JOptionPane.showMessageDialog(d, "Invalid quantity.");
return;
}
String sql = "{CALL sp_add_to_cart(?, ?, ?)}";

```

```

try (Connection c = Main.getConnection();
CallableStatement cs = c.prepareCall(sql)) {

```

```

cs.setInt(1, userId);
cs.setInt(2, p.id);
cs.setInt(3, qty);

```

```
cs.execute();
```

```
JOptionPane.showMessageDialog(d, "Added to cart (qty=" + qty + ").");
```

```
updateCartButtonCount();
```

```
} catch (SQLException ex) {
```

```
JOptionPane.showMessageDialog(d, "Cart DB Error: " + ex.getMessage()); }
```

```
});
```

```
btnBuy.addActionListener(e-> JOptionPane.showMessageDialog(d, "Buy flow coming soon!"));
```

```
submitFeedbackBtn.addActionListener(e -> {
```

```
String ratingStr = (String) ratingBox.getSelectedItem();
```

```
String comment = commentArea.getText().trim();
```

```
if (ratingStr == null || comment.isEmpty()) {
```

```
JOptionPane.showMessageDialog(d, "Please select a rating and write a comment."); return;
```

```
}
```

```
int rating = Integer.parseInt(ratingStr.substring(0, 1));
```

```
String sql = "INSERT INTO feedback (user_id, product_id, rating, comment,  
feedback_date) VALUES (?, ?, ?, ?, NOW()) " +
```

```
"ON DUPLICATE KEY UPDATE rating = ?, comment = ?, feedback_date = NOW()";
```

```
try (Connection c = Main.getConnection(); PreparedStatement ps =  
c.prepareStatement(sql)) {
```

```
ps.setInt(1, userId);
```

```
ps.setInt(2, p.id);
```

```
ps.setInt(3, rating);
```

```
ps.setString(4, comment);
```

```
ps.setInt(5, rating);
```

```
ps.setString(6, comment);
```

```
ps.executeUpdate();
```

```
sendFeedbackNotification(p.companyId, rating, p.name);
```

```
JOptionPane.showMessageDialog(d, "Feedback submitted successfully!");
```

```
commentArea.setText("");
```

```
existingFeedbackPanel.removeAll();
```

```
loadProductFeedback(existingFeedbackPanel, p.id);
```

```
existingFeedbackPanel.revalidate();
```

```
existingFeedbackPanel.repaint();
```

```
} catch (SQLException ex) {
```

```
JOptionPane.showMessageDialog(d, "Feedback DB Error: " + ex.getMessage()); }
```

```
});
```

```
new Thread() -> {
```

```
String sql = "SELECT p.product_id, p.company_id, p.product_name, p.category, p.price,  
p.rating, p.image_path, c.company_name, p.description " +
```

```
"FROM product p LEFT JOIN company c ON p.company_id=c.company_id " +  
"WHERE p.category=? AND p.product_id != ? ORDER BY RAND() LIMIT 5";
```

```
try (Connection c = Main.getConnection(); PreparedStatement ps =  
c.prepareStatement(sql)) {
```

```
ps.setString(1, p.category);
```

```
ps.setInt(2, p.id);
```

```
try (ResultSet rs = ps.executeQuery()) {
```

```
while (rs.next()) {
```

```

ProductModel p2 = new ProductModel();
p2.id = rs.getInt("product_id");
p2.companyId = rs.getInt("company_id");
p2.name = rs.getString("product_name");
p2.category = rs.getString("category");
p2.price = rs.getDouble("price");
p2.rating = rs.getDouble("rating");
p2.imagePath = rs.getString("image_path");
p2.companyName = rs.getString("company_name");
p2.description = rs.getString("description");

```

```

SwingUtilities.invokeLater(() -> {
    relatedItemsPanel.add(createRelatedProductButton(p2, d));
    relatedItemsPanel.revalidate();
    relatedItemsPanel.repaint();
});
}
}
} catch (SQLException ex) { }
}).start();

```

```

new Thread(() -> {
    loadProductFeedback(existingFeedbackPanel, p.id);
}).start();

d.setVisible(true);
}

```

```

private void sendFeedbackNotification(int companyId, int rating, String productName) {
    new Thread(() -> {

```

```

String message = String.format("You received a %d-star review for '%s'.", rating,
productName);

String sql = "INSERT INTO notification (company_id, message, date_sent) VALUES (?, ?,
NOW())";

try (Connection conn = Main.getConnection();
PreparedStatement ps = conn.prepareStatement(sql)) {

    ps.setInt(1, companyId);
    ps.setString(2, message);
    ps.executeUpdate();

} catch (SQLException e) {

    System.err.println("Failed to send feedback notification: " + e.getMessage()); }

}).start();
}

```

```

private void loadProductFeedback(JPanel panel, int productId) {
    String sql = "SELECT f.rating, f.comment, f.feedback_date, u.name " +
    "FROM feedback f JOIN user u ON f.user_id = u.user_id " + "WHERE
    f.product_id = ? ORDER BY f.feedback_date DESC";

    try (Connection c = Main.getConnection(); PreparedStatement ps =
    c.prepareStatement(sql)) {
        ps.setInt(1, productId);

        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.isBeforeFirst()) {
                SwingUtilities.invokeLater(() -> {
                    JLabel noFeedback = new JLabel("No reviews yet. Be the first!");
                    noFeedback.setFont(new Font("SansSerif", Font.ITALIC, 12));

```

```

noFeedback.setBorder(new EmptyBorder(10, 10, 10, 10));

panel.add(noFeedback);

panel.revalidate();

panel.repaint();

});

return;

}

while (rs.next()) {

String name = rs.getString("name");

int rating = rs.getInt("rating");

String comment = rs.getString("comment");

Timestamp date = rs.getTimestamp("feedback_date");

SwingUtilities.invokeLater(() -> {

panel.add(createFeedbackEntryPanel(name, rating, comment, date));

panel.revalidate();

panel.repaint();

});

}

}

} catch (SQLException ex) {

}

}

}

```

```

private JPanel createFeedbackEntryPanel(String name, int rating, String comment,
java.util.Date date) {

JPanel entry = new JPanel(new BorderLayout(5, 5));

entry.setBackground(COLOR_JASMINE_WHITE);

entry.setBorder(BorderFactory.createCompoundBorder(

BorderFactory.createMatteBorder(0, 0, 1, 0, Color.LIGHT_GRAY), new

```

```
EmptyBorder(10, 10, 10, 10)
```

```
));
```

```
String dateStr = new SimpleDateFormat("dd-MMM-yyyy").format(date);
```

```
StringBuilder stars = new StringBuilder();
```

```
for(int i=0; i<5; i++) {
```

```
stars.append(i < rating ? "★" : "☆");
```

```
}
```

```
JLabel header = new JLabel(String.format("<html><b>%s</b> <font  
color='orange'>%s</font> <font color='gray'>(%s)</font></html>", name,  
stars.toString(), dateStr));
```

```
header.setFont(new Font("SansSerif", Font.PLAIN, 13));
```

```
entry.add(header, BorderLayout.NORTH);
```

```
JTextArea commentText = new JTextArea(comment);
```

```
commentText.setLineWrap(true);
```

```
commentText.setWrapStyleWord(true);
```

```
commentText.setEditable(false);
```

```
commentText.setOpaque(false);
```

```
commentText.setFont(new Font("SansSerif", Font.PLAIN, 12));
```

```
entry.add(commentText, BorderLayout.CENTER);
```

```
entry.setMaximumSize(new Dimension(Integer.MAX_VALUE,  
(int)entry.getPreferredSize().getHeight()));
```

```
return entry;
```

```
}
```

```
private JButton createRelatedProductButton(ProductModel p, JDialog parentDialog) {
```

```
JButton btn = new JButton();
```

```

btn.setLayout(new BorderLayout(5, 5));

btn.setOpaque(false);

btn.setBorder(BorderFactory.createEmptyBorder(5,5,5,5));

String name = p.name.length() > 15 ? p.name.substring(0, 15) + "...": p.name;

JLabel nameLbl = new JLabel(name);

nameLbl.setFont(new Font("SansSerif", Font.BOLD, 12));

JLabel priceLbl = new JLabel(String.format("₹%.2f", p.price));

priceLbl.setFont(new Font("SansSerif", Font.PLAIN, 11));

btn.add(new JLabel(loadImageIcon(p.imagePath, 80, 80)), BorderLayout.CENTER);

btn.add(nameLbl, BorderLayout.SOUTH);

btn.add(priceLbl, BorderLayout.NORTH);

btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

btn.setToolTipText(p.name);

btn.addActionListener(e -> {

    parentDialog.dispose();

    openProductDetailDialog(p);
});

return btn;

}

private void openWishlistDialog() {

    JDialog d = new JDialog(this, "Your Wishlist", true);

    d.setSize(720, 520);

    d.setLocationRelativeTo(this);

    d.getContentPane().setBackground(COLOR_PRIMARY_BG);

```

```

DefaultListModel<ProductModel> listModel = new DefaultListModel<>();

JList<ProductModel> list = new JList<>(listModel);

list.setCellRenderer(new ProductRenderer());

try (Connection c = Main.getConnection()) {

    String sql = "SELECT p.product_id, p.company_id, p.product_name, p.category, p.price,
p.rating, p.image_path, c.company_name, p.description " +

    "FROM wishlist w JOIN product p ON w.product_id=p.product_id LEFT JOIN company c
ON p.company_id=c.company_id " +

    "WHERE w.user_id=?";

    try (PreparedStatement ps = c.prepareStatement(sql)) {

        ps.setInt(1, userId);

        try (ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                ProductModel p = new ProductModel();

                p.id = rs.getInt("product_id");

                p.companyId = rs.getInt("company_id");

                p.name = rs.getString("product_name");

                p.category = rs.getString("category");

                p.price = rs.getDouble("price");

                p.rating = rs.getDouble("rating");

                p.imagePath = rs.getString("image_path");

                p.companyName = rs.getString("company_name");
                p.description = rs.getString("description");

                listModel.addElement(p);

            }

        }

    } catch (SQLException ex) { JOptionPane.showMessageDialog(this, "Wishlist load error: " +
ex.getMessage()); }
}

```

```

JButton remove = new JButton("Remove Selected");

remove.addActionListener(e -> {

    ProductModel sel = list.getSelectedValue();

    if (sel == null) { JOptionPane.showMessageDialog(d, "Select item first."); return; } try

    (Connection c = Main.getConnection()) {

        String del = "DELETE FROM wishlist WHERE user_id=? AND product_id=? LIMIT 1"; try

        (PreparedStatement ps = c.prepareStatement(del)) {

            ps.setInt(1, userId); ps.setInt(2, sel.id);

            ps.executeUpdate();

            listModel.removeElement(sel);

        }

    } catch (SQLException ex) { JOptionPane.showMessageDialog(d, "Remove error: " +

    ex.getMessage()); }

});

```

```

d.add(new JScrollPane(list), BorderLayout.CENTER);

JPanel b = new JPanel(new FlowLayout(FlowLayout.RIGHT)); b.setOpaque(false);
b.add(remove);

d.add(b, BorderLayout.SOUTH);

d.setVisible(true);

}

```

```

private void openCartDialog() {

    JDialog d = new JDialog(this, "Your Cart", true);
    d.setSize(780, 560);

    d.setLocationRelativeTo(this);

    d.getContentPane().setBackground(COLOR_PRIMARY_BG);
}

```

```

DefaultListModel<CartItem> model = new DefaultListModel<>();

JList<CartItem> list = new JList<>(model);

```

```
list.setCellRenderer(new CartRenderer());
```

```
try (Connection c = Main.getConnection()) {
```

```
String sql = "SELECT c.quantity, p.product_id, p.company_id, p.product_name, p.category,  
p.price, p.rating, p.image_path, comp.company_name, p.description " +
```

```
"FROM cart c JOIN product p ON c.product_id=p.product_id LEFT JOIN company comp ON  
p.company_id=comp.company_id " +
```

```
"WHERE c.user_id=?";
```

```
try (PreparedStatement ps = c.prepareStatement(sql)) {
```

```
ps.setInt(1, userId);
```

```
try (ResultSet rs = ps.executeQuery()) {
```

```
while (rs.next()) {
```

```
ProductModel p = new ProductModel();
```

```
p.id = rs.getInt("product_id");
```

```
p.companyId = rs.getInt("company_id");
```

```
p.name = rs.getString("product_name");
```

```
p.category = rs.getString("category");
```

```
p.price = rs.getDouble("price");
```

```
p.rating = rs.getDouble("rating");
```

```
p.imagePath = rs.getString("image_path");
```

```
p.companyName = rs.getString("company_name");
```

```
p.description = rs.getString("description");
```

```
CartItem it = new CartItem();
```

```
it.quantity = rs.getInt("quantity");
```

```
it.product = p;
```

```
model.addElement(it);
```

```
}
```

```
}
```

```
}
```

```
} catch (SQLException ex) { JOptionPane.showMessageDialog(this, "Cart load error: " +
```

```
ex.getMessage()); }
```

```
JLabel lblTotal = new JLabel("Grand Total: ₹ " + String.format("%.2f",  
computeCartTotal(model)));
```

```
lblTotal.setFont(new Font("SansSerif", Font.BOLD, 14));
```

```
JButton remove = new JButton("Remove Selected");
```

```
JButton checkout = new JButton("Checkout");
```

```
remove.addActionListener(e -> {
```

```
CartItem sel = list.getSelectedValue();
```

```
if (sel == null) { JOptionPane.showMessageDialog(d, "Select item first."); return; } try
```

```
(Connection c = Main.getConnection()) {
```

```
String del = "DELETE FROM cart WHERE user_id=? AND product_id=?"; try
```

```
(PreparedStatement ps = c.prepareStatement(del)) {
```

```
ps.setInt(1, userId);
```

```
ps.setInt(2, sel.product.id);
```

```
ps.executeUpdate();
```

```
model.removeElement(sel);
```

```
lblTotal.setText("Grand Total: ₹ " + String.format("%.2f", computeCartTotal(model)));
```

```
updateCartButtonCount();
```

```
}
```

```
} catch (SQLException ex) { JOptionPane.showMessageDialog(d, "Remove error: " +  
ex.getMessage()); }
```

```
});
```

```
checkout.addActionListener(e -> {
```

```
if (model.isEmpty()) { JOptionPane.showMessageDialog(d, "Cart is empty."); return; }
```

```
try (Connection c = Main.getConnection()) {
```

```
String del = "DELETE FROM cart WHERE user_id=?";
```

```
try (PreparedStatement ps = c.prepareStatement(del)) {
```

```
ps.setInt(1, userId); ps.executeUpdate();
```

```

model.clear();

lblTotal.setText("Grand Total: ₹ 0.00");

JOptionPane.showMessageDialog(d, "Order placed! (demo)");

updateCartButtonCount();

}

} catch (SQLException ex) { JOptionPane.showMessageDialog(d, "Checkout error: " +
ex.getMessage()); }

});

```

```

JPanel bottom = new JPanel(new BorderLayout());

bottom.setOpaque(false);

JPanel rightBtns = new JPanel(new FlowLayout(FlowLayout.RIGHT));
rightBtns.setOpaque(false);

rightBtns.add(remove); rightBtns.add(checkout);

bottom.add(lblTotal, BorderLayout.WEST);

bottom.add(rightBtns, BorderLayout.EAST);

```

```

d.add(new JScrollPane(list), BorderLayout.CENTER);

d.add(bottom, BorderLayout.SOUTH);

d.setVisible(true);

}

```

```

private double computeCartTotal(DefaultListModel<CartItem> model) {

double t = 0.0;

for (int i = 0; i < model.size(); i++) t += model.get(i).product.price * model.get(i).quantity;

return t;

}

private Icon loadImageIcon(String path, int w, int h) {

if (path != null && !path.trim().isEmpty()) {

try {

File f = new File(path);

```

```

if (!f.exists()) f = new File(System.getProperty("user.dir"), path); if
(f.exists()) {

    BufferedImage bi = ImageIO.read(f);
    return new ImageIcon(bi.getScaledInstance(w, h, Image.SCALE_SMOOTH)); }

} catch (IOException ignored) {}

}

BufferedImage img = new BufferedImage(w, h, BufferedImage.TYPE_INT_ARGB);
Graphics2D g = img.createGraphics();

g.setColor(new Color(240,240,240));
g.fillRect(0,0,w,h);

g.setColor(new Color(180,180,180));
g.drawString("No Image", 10, h/2);
g.dispose();

return new ImageIcon(img);

}

```

```

static class ProductModel {

    int id, companyId;

    String name, category, description, imagePath, companyName;

    double price, rating;

    public String toString(){ return name + " — ₹" + price; }

}

```

```

static class CartItem {

    int quantity;

    ProductModel product;

    public String toString(){ return product.name + " x" + quantity; } }

```

```

class ProductRenderer extends JPanel implements ListCellRenderer<ProductModel> {

    private final JLabel icon = new JLabel();

```

```

private final JLabel title = new JLabel();

private final JLabel price = new JLabel();

private final JLabel company = new JLabel();


public ProductRenderer() {

setLayout(new BorderLayout(10,10));

setBorder(new EmptyBorder(8,8,8,8));

setBackground(COLOR_JASMINE_WHITE);

icon.setPreferredSize(new Dimension(90,90));

add(icon, BorderLayout.WEST);

JPanel center = new JPanel();

center.setOpaque(false);

center.setLayout(new BoxLayout(center, BoxLayout.Y_AXIS));

title.setFont(new Font("SansSerif", Font.BOLD, 15));

price.setFont(new Font("SansSerif", Font.BOLD, 13));

price.setForeground(new Color(39,174,96));

company.setFont(new Font("SansSerif", Font.ITALIC, 12));

company.setForeground(COLOR_TEXT_DIM);

center.add(title); center.add(price); center.add(company);

add(center, BorderLayout.CENTER);

}


public Component getListCellRendererComponent(JList<? extends ProductModel> list,
ProductModel p, int idx, boolean sel, boolean focus) {

icon.setIcon(loadImageIcon(p.imagePath, 90, 90));

title.setText(p.name);

price.setText("₹ " + String.format("%.2f", p.price));
company.setText(p.companyName == null ? "" : p.companyName + " • " +
(p.category==null?"":p.category));

if (sel) setBackground(new Color(220, 245, 255)); else
setBackground(COLOR_JASMINE_WHITE);

return this;

}

```

```
}
```

```
class CartRenderer extends JPanel implements ListCellRenderer<CartItem> {  
    private final JLabel icon = new JLabel();  
    private final JLabel title = new JLabel();  
    private final JLabel qty = new JLabel();  
    private final JLabel total = new JLabel();  
    public CartRenderer() {  
        setLayout(new BorderLayout(10,10));  
        setBorder(new EmptyBorder(8,8,8,8));  
        setBackground(COLOR_JASMINE_WHITE);  
        icon.setPreferredSize(new Dimension(90,90));  
        add(icon, BorderLayout.WEST);  
        JPanel center = new JPanel();  
        center.setOpaque(false);  
        center.setLayout(new BoxLayout(center, BoxLayout.Y_AXIS));  
        title.setFont(new Font("SansSerif", Font.BOLD, 15));  
        qty.setFont(new Font("SansSerif", Font.PLAIN, 13));  
        total.setFont(new Font("SansSerif", Font.BOLD, 13));  
        total.setForeground(new Color(39,174,96));  
        center.add(title); center.add(qty); center.add(total);  
        add(center, BorderLayout.CENTER);  
    }  
  
    public Component getListCellRendererComponent(JList<? extends CartItem> list, CartItem it,  
    int idx, boolean sel, boolean focus) {  
        icon.setIcon(loadImageIcon(it.product.imagePath, 90, 90));  
        title.setText(it.product.name);  
        qty.setText("Qty: " + it.quantity);  
        total.setText("Total: ₹ " + String.format("%.2f", it.product.price * it.quantity));  
        if (sel) setBackground(new Color(220, 245, 255)); else  
        setBackground(COLOR_JASMINE_WHITE);  
    }  
}
```

```
return this;
```

```
}
```

```
}
```

```
}
```

```
// COMPANY PORTAL CLASSES (MERGED)
```

```
static class Company_portal {
```

```
// COMPANY LOGIN
```

```
public static boolean companyLogin(Connection conn, String email, String password) throws  
SQLException {
```

```
String sql = "SELECT company_id, company_name, owner_name, category, state FROM  
company WHERE email = ? AND password = ?";
```

```
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
pstmt.setString(1, email);
```

```
pstmt.setString(2, password);
```

```
try (ResultSet rs = pstmt.executeQuery()) {
```

```
if (rs.next()) {
```

```
System.out.println("\nWelcome, " + rs.getString("company_name")); return
```

```
true;
```

```
}
```

```
return false;
```

```
}
```

```
}
```

```
}
```

```
// 1 PRODUCT ANALYTICS
```

```
public static void viewProductAnalytics(Connection conn, int companyId) throws SQLException {
```

```
String sql = "SELECT p.product_name, COUNT(v.view_id) AS total_views " +
```

```
"FROM product p LEFT JOIN view v ON p.product_id = v.product_id " + "WHERE
```

```
p.company_id = ? " +
```

```
"GROUP BY p.product_id, p.product_name " +  
"ORDER BY total_views DESC";
```

```
Map<String, Integer> dataset = new LinkedHashMap<>();
```

```
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {  
    pstmt.setInt(1, companyId);  
    try (ResultSet rs = pstmt.executeQuery()) {  
        while (rs.next()) {  
            String name = rs.getString("product_name");  
            int views = rs.getInt("total_views");  
            dataset.put(name, views);  
        }  
    }  
}
```

```
if (dataset.size() > 0) {  
    SimpleBarChart chartPanel = new SimpleBarChart(dataset);  
    chartPanel.setPreferredSize(new Dimension(800, 600));  
  
    JFrame frame = new JFrame("Product Analytics - Views");  
    frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
```

```
    JScrollPane scrollPane = new JScrollPane(chartPanel);  
    frame.add(scrollPane);
```

```
    frame.setSize(800, 600);  
    frame.setLocationRelativeTo(null);  
    frame.setAlwaysOnTop(true);  
    frame.setVisible(true);  
} else {
```

```
JOptionPane.showMessageDialog(null, "No product views data available."); }  
}
```

```
// 2 SALES ANALYTICS
```

```
public static void viewSalesAnalytics(Connection conn, int companyId) throws SQLException {  
    System.out.println("\n--- SALES ANALYTICS (from User Carts) ---");
```

```
    String sql = "SELECT p.product_name, " +  
        "COALESCE(SUM(c.quantity), 0) as total_in_carts, " +  
        "COALESCE(SUM(c.quantity * p.price), 0) as total_revenue_in_carts " + "FROM  
product p " +  
        "LEFT JOIN cart c ON p.product_id = c.product_id " +  
        "WHERE p.company_id = ? " +  
        "GROUP BY p.product_id, p.product_name " +  
        "ORDER BY total_revenue_in_carts DESC";
```

```
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {  
        pstmt.setInt(1, companyId);  
        ResultSet rs = pstmt.executeQuery();
```

```
        System.out.println("Product Name\t\tUnits in Carts\tRevenue in Carts");  
        System.out.println("-----");
```

```
        double totalCompanyRevenue = 0;  
        int totalUnitsSold = 0;
```

```
        while (rs.next()) {  
            String productName = rs.getString("product_name");  
            int unitsSold = rs.getInt("total_in_carts");  
            double revenue = rs.getDouble("total_revenue_in_carts");
```

```
System.out.printf("%-20s\t%d\t\t₹%.2f\n", productName, unitsSold, revenue);
```

```
totalUnitsSold += unitsSold;
```

```
totalCompanyRevenue += revenue;
```

```
}
```

```
System.out.println("-----");
```

```
System.out.printf("TOTAL (in Carts):\t\t%d\t\t₹%.2f\n", totalUnitsSold,  
totalCompanyRevenue);
```

```
} catch (SQLException e) {
```

```
System.out.println("Sales data not available yet.");
```

```
}
```

```
}
```

```
// 3 MANAGE PRODUCTS
```

```
public static void addProduct(Connection conn, int companyId, String name, String category,  
double price, String description, String imagePath) throws SQLException {
```

```
String sql = "INSERT INTO product (company_id, product_name, category, price, description,  
image_path, rating) VALUES (?, ?, ?, ?, ?, ?, 0.0)";
```

```
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
pstmt.setInt(1, companyId);
```

```
pstmt.setString(2, name);
```

```
pstmt.setString(3, category);
```

```
pstmt.setDouble(4, price);
```

```
pstmt.setString(5, description);
```

```
pstmt.setString(6, imagePath);
```

```
int rows = pstmt.executeUpdate();
```

```
if (rows > 0) {
```

```
System.out.println("Product added successfully!");
```

```
addNotification(conn, companyId, "New product '" + name + "' added successfully!"); } else {
```

```

System.out.println("Failed to add product.");
}
}
}

```

```

public static void updateProduct(Connection conn, int companyId, int productId, String
newName, String newCategory, double newPrice, String newDesc, String newImg)
throws SQLException {

```

```

String sql = "UPDATE product SET product_name = ?, category = ?, price = ?, description = ?,
image_path = ? WHERE product_id = ? AND company_id = ?";

```

```

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

```

```

    pstmt.setString(1, newName);

```

```

    pstmt.setString(2, newCategory);

```

```

    pstmt.setDouble(3, newPrice);

```

```

    pstmt.setString(4, newDesc);

```

```

    pstmt.setString(5, newImg);

```

```

    pstmt.setInt(6, productId);

```

```

    pstmt.setInt(7, companyId);

```

```

    int rows = pstmt.executeUpdate();

```

```

    if (rows > 0) {

```

```

        System.out.println("Product updated successfully!");

```

```

        addNotification(conn, companyId, "Product "" + newName + "" updated successfully!"); } else {

```

```

        System.out.println("Update failed! Product not found or you don't have permission."); }

```

```

    }

```

```

}

```

```

public static void deleteProduct(Connection conn, int companyId, int productId) throws
SQLException {

```

```

String productName = getProduct(conn, productId, companyId);

```

```

String sql = "DELETE FROM product WHERE product_id = ? AND company_id = ?"; try

```

```

(PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setInt(1, productId);
    pstmt.setInt(2, companyId);

    int rows = pstmt.executeUpdate();
    if (rows > 0) {
        System.out.println("Product deleted successfully!");
        if (productName != null) {
            addNotification(conn, companyId, "Product '" + productName + "' deleted!"); }
        } else {
            System.out.println("Product not found or you don't have permission!"); }
        }
    }

// 4 VIEW NOTIFICATIONS

public static void viewNotifications(Connection conn, int companyId) throws SQLException {

    System.out.println("\n--- COMPANY NOTIFICATIONS ---");

    String sql = "SELECT notification_no, message, date_sent FROM notification WHERE
company_id = ? ORDER BY date_sent DESC LIMIT 10";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, companyId);
        ResultSet rs = pstmt.executeQuery();

        boolean hasNotifications = false;

        while (rs.next()) {

            hasNotifications = true;

            System.out.println(" [" + rs.getTimestamp("date_sent") + "] " +
rs.getString("message"));
        }
    }
}

```

```

if (!hasNotifications) {
    System.out.println("No notifications found.");
}
}
}

// 5 COMPANY PROFILE

public static void viewCompanyProfile(Connection conn, int companyId) throws SQLException {
    String sql = "SELECT * FROM company WHERE company_id = ?";
    PreparedStatement pstmt = conn.prepareStatement(sql);
    pstmt.setInt(1, companyId);
    ResultSet rs = pstmt.executeQuery();

    if (rs.next()) {
        System.out.println("\n--- COMPANY PROFILE ---");
        System.out.println("ID: " + rs.getInt("company_id"));
        System.out.println("Name: " + rs.getString("company_name"));
        System.out.println("Owner: " + rs.getString("owner_name"));
        System.out.println("Email: " + rs.getString("email"));
        System.out.println("Phone: " + rs.getString("phone"));
        System.out.println("Category: " + rs.getString("category"));
        System.out.println("Address: " + rs.getString("street") + ", " + rs.getString("city") + " - " + rs.getString("postal_code"));
        System.out.println("State: " + rs.getString("state"));
        System.out.println("Verification Status: " + (rs.getBoolean("verified") ? "Verified " : "Pending "));
    }
}

public static int getCompanyId(Connection conn, String email) throws SQLException {

```

```

String sql = "SELECT company_id FROM company WHERE email = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);

pstmt.setString(1, email);

ResultSet rs = pstmt.executeQuery();

if (rs.next()) return rs.getInt(1);

return -1;

}

```

```

private static void addNotification(Connection conn, int companyId, String message) throws
SQLException {

String sql = "INSERT INTO notification (company_id, message, date_sent) VALUES (?, ?,
NOW())";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

pstmt.setInt(1, companyId);

pstmt.setString(2, message);

pstmt.executeUpdate();

}

}

```

```

private static String getProductName(Connection conn, int productId, int companyId) throws
SQLException {

String sql = "SELECT product_name FROM product WHERE product_id = ? AND company_id =
?";

PreparedStatement pstmt = conn.prepareStatement(sql);

pstmt.setInt(1, productId);
pstmt.setInt(2, companyId);

ResultSet rs = pstmt.executeQuery();

return rs.next() ? rs.getString("product_name") : null;

}

}

```

```

static class CompanyPortalGUI {

```

```
private JFrame frame;

private JTextField emailField;

private JPasswordField passwordField;


private int currentCompanyId;

private String currentEmail;

private String currentCompanyName;


public CompanyPortalGUI() {

    createLoginWindow();

}


// LOGIN WINDOW

private void createLoginWindow() {

    frame = new JFrame("Company Portal - Login");

    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    frame.setSize(450, 300);

    frame.setLayout(new BorderLayout(10, 10));


    JLabel headerLabel = new JLabel("Company Login", SwingConstants.CENTER);

    headerLabel.setFont(new Font("Arial", Font.BOLD, 20));

    headerLabel.setForeground(new Color(0, 100, 200));

    JPanel mainPanel = new JPanel(new GridLayout(3, 2, 10, 10));

    mainPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));


    JLabel emailLabel = new JLabel("Email:");

    emailLabel.setFont(new Font("Arial", Font.BOLD, 14));

    JLabel passLabel = new JLabel("Password:");

    passLabel.setFont(new Font("Arial", Font.BOLD, 14));
```

```
emailField = new JTextField();  
passwordField = new JPasswordField();
```

```
mainPanel.add(emailLabel);  
mainPanel.add(emailField);  
mainPanel.add(passLabel);  
mainPanel.add(passwordField);
```

```
JPanel buttonPanel = new JPanel(new FlowLayout());  
JButton loginButton = new JButton("Login");  
loginButton.setBackground(new Color(0, 150, 0));  
loginButton.setForeground(Color.WHITE);  
loginButton.setFont(new Font("Arial", Font.BOLD, 14));  
loginButton.setOpaque(true);  
loginButton.setBorderPainted(false);
```

```
JButton backButton = new JButton(" Back");  
backButton.setBackground(new Color(200, 0, 0));  
backButton.setForeground(Color.WHITE);  
backButton.setFont(new Font("Arial", Font.BOLD, 14));  
backButton.setOpaque(true);  
backButton.setBorderPainted(false);  
loginButton.addActionListener(e -> handleLogin());  
backButton.addActionListener(e -> {  
    frame.dispose();  
    new WelcomeFrame();  
});
```

```
buttonPanel.add(loginButton);  
buttonPanel.add(backButton);
```

```

frame.add(headerLabel, BorderLayout.NORTH);

frame.add(mainPanel, BorderLayout.CENTER);

frame.add(buttonPanel, BorderLayout.SOUTH);


frame.setLocationRelativeTo(null);

frame.setVisible(true);

}


// LOGIN HANDLER

private void handleLogin() {

String email = emailField.getText().trim();

String password = new String(passwordField.getPassword());


if (email.isEmpty() || password.isEmpty()) {

JOptionPane.showMessageDialog(frame, "Please enter both email and password!");

return;

}


try (Connection conn = Main.getConnection()) {

if (Company_portal.companyLogin(conn, email, password)) {

this.currentCompanyId = Company_portal.getCompanyId(conn, email);

this.currentCompanyName = getCompanyName(conn, currentCompanyId);

JOptionPane.showMessageDialog(frame, "Login Successful! Welcome " +
this.currentCompanyName);

frame.dispose();

currentEmail = email;

showDashboard();

} else {

JOptionPane.showMessageDialog(frame, "Invalid Credentials!"); }
}

```

```

    } catch (Exception ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(frame, "Error: " + ex.getMessage());
    }

    // Helper to get name
    private String getCompanyName(Connection conn, int companyId) throws SQLException {
        String sql = "SELECT company_name FROM company WHERE company_id = ?";
        try(PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, companyId);
            try(ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getString("company_name");
                }
            }
        }
        return "Company";
    }

    // DASHBOARD WINDOW
    private void showDashboard() {
        JFrame dashboard = new JFrame("Company Dashboard: " + currentCompanyName);
        dashboard.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        dashboard.setSize(700, 500);
        dashboard.setLayout(new BorderLayout(10, 10));

        JLabel headerLabel = new JLabel("Company Dashboard", SwingConstants.CENTER);
        headerLabel.setFont(new Font("Arial", Font.BOLD, 24));
        headerLabel.setForeground(new Color(0, 100, 200));
        headerLabel.setBorder(BorderFactory.createEmptyBorder(20, 0, 20, 0));
    }

```

```
JPanel buttonPanel = new JPanel(new GridLayout(3, 2, 15, 15));  
buttonPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
```

```
JButton analyticsBtn = createStyledButton(" Product Analytics", new Color(70, 130, 180));  
JButton salesBtn = createStyledButton(" Sales Analytics", new Color(34, 139, 34)); JButton  
manageBtn = createStyledButton(" Manage Products", new Color(255, 140, 0)); JButton  
notifyBtn = createStyledButton(" Notifications", new Color(138, 43, 226)); JButton profileBtn =  
createStyledButton(" Company Profile", new Color(47, 79, 79)); JButton logoutBtn =  
createStyledButton(" Logout", new Color(220, 20, 60));
```

```
analyticsBtn.addActionListener(e -> showProductAnalytics());  
salesBtn.addActionListener(e -> showSalesAnalytics());  
manageBtn.addActionListener(e -> showManageProducts());  
notifyBtn.addActionListener(e -> showNotifications());  
profileBtn.addActionListener(e -> showCompanyProfile());  
logoutBtn.addActionListener(e -> {  
    dashboard.dispose();  
    new WelcomeFrame();  
});  
buttonPanel.add(analyticsBtn);  
buttonPanel.add(salesBtn);  
buttonPanel.add(manageBtn);  
buttonPanel.add(notifyBtn);  
buttonPanel.add(profileBtn);  
buttonPanel.add(logoutBtn);
```

```
dashboard.add(headerLabel, BorderLayout.NORTH);  
dashboard.add(buttonPanel, BorderLayout.CENTER);
```

```

dashboard.setLocationRelativeTo(null);

dashboard.setVisible(true);

}

// CREATE STYLED BUTTONS

private JButton createStyledButton(String text, Color color) {

JButton button = new JButton(text);

button.setFont(new Font("Arial", Font.BOLD, 16));

button.setForeground(Color.WHITE);

button.setBackground(color);

button.setOpaque(true);

button.setBorderPainted(false);


button.setFocusPainted(false);

button.setPreferredSize(new Dimension(200, 60));

button.addMouseListener(new java.awt.event.MouseAdapter() {

public void mouseEntered(java.awt.event.MouseEvent evt) {

button.setBackground(color.darker());

}

public void mouseExited(java.awt.event.MouseEvent evt) {

button.setBackground(color);

}

});

return button;

}

private void showProductAnalytics() {

try (Connection conn = Main.getConnection()) {

// This now opens a JDialog with 3 tabs

```

```

createAnalyticsDialog(conn, currentCompanyId);
} catch (Exception ex) {
OptionPane.showMessageDialog(null, "Error displaying analytics: " + ex.getMessage()); }
}

private void createAnalyticsDialog(Connection conn, int companyId) throws SQLException {
JDialog dialog = new JDialog(frame, "Product Analytics Dashboard", true); dialog.setSize(800,
600);

dialog.setLocationRelativeTo(null);

JTabbedPane tabbedPane = new JTabbedPane();

// --- Tab 1: Product Views ---
Map<String, Integer> viewData = getProductViewData(conn, companyId); if
(!viewData.isEmpty()) {
    Main.SimpleBarChart chartPanel = new Main.SimpleBarChart(viewData);
chartPanel.setPreferredSize(new Dimension(viewData.size() * 150, 500));
JScrollPane chartScroll = new JScrollPane(chartPanel);

tabbedPane.addTab("Product Views", chartScroll);
} else {
tabbedPane.addTab("Product Views", new JLabel("No product view data available.")); }

// --- Tab 2: Wishlist Popularity ---
String wishlistData = getWishlistData(conn, companyId);
JTextArea wishlistArea = new JTextArea(wishlistData);
wishlistArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
wishlistArea.setEditable(false);
tabbedPane.addTab("Wishlist Popularity", new JScrollPane(wishlistArea));

// --- Tab 3: Product Ratings---
String ratingData = getRatingData(conn, companyId);

```

```
JTextArea ratingArea = new JTextArea(ratingData);
ratingArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
ratingArea.setEditable(false);
tabbedPane.addTab("Product Ratings", new JScrollPane(ratingArea));
```

```
dialog.add(tabbedPane);
dialog.setVisible(true);
}
```

```
// NEW: Query 1 for Analytics
```

```
private Map<String, Integer> getProductViewData(Connection conn, int companyId) throws
SQLException {
```

```
    Map<String, Integer> dataset = new LinkedHashMap<>();
```

```
    String sql = "SELECT p.product_name, COUNT(v.view_id) AS total_views " +
```

```
"FROM product p LEFT JOIN view v ON p.product_id = v.product_id " + "WHERE
p.company_id = ? " +
```

```
"GROUP BY p.product_id, p.product_name " +
```

```
"ORDER BY total_views DESC";
```

```
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
        pstmt.setInt(1, companyId);
```

```
        try (ResultSet rs = pstmt.executeQuery()) {
```

```
            while (rs.next()) {
```

```
                String name = rs.getString("product_name");
```

```
                int views = rs.getInt("total_views");
```

```
                dataset.put(name, views);
```

```
            }
```

```
        }
```

```
    }
```

```
    return dataset;
```

```
}
```

```

// NEW: Query 2 for Analytics (Wishlist)

private String getWishlistData(Connection conn, int companyId) throws SQLException {
    StringBuilder sb = new StringBuilder();

    sb.append("=== PRODUCT WISHLIST POPULARITY ===\n\n");

    String sql = "SELECT p.product_name, COUNT(w.user_id) AS wishlist_adds " + "FROM
product p LEFT JOIN wishlist w ON p.product_id = w.product_id " + "WHERE
p.company_id = ? " +
    "GROUP BY p.product_id, p.product_name " +
    "ORDER BY wishlist_adds DESC";

    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, companyId);

        ResultSet rs = pstmt.executeQuery();

        sb.append(String.format("%-30s %-10s\n", "Product Name", "Times Added to Wishlist"));
        sb.append("-----\n");

        boolean hasData = false;
        while (rs.next()) {
            hasData = true;

            sb.append(String.format("%-30s %-10d\n", rs.getString("product_name"),
rs.getInt("wishlist_adds")));
        }

        if (!hasData) {
            sb.append("No products have been added to any wishlists yet.");
        }

        return sb.toString();
    }
}

```

```

// NEW: Query 3 for Analytics (Ratings)

private String getRatingData(Connection conn, int companyId) throws SQLException {

StringBuilder sb = new StringBuilder();

sb.append("=== PRODUCT RATINGS & REVIEWS ===\n\n");


String sql = "SELECT p.product_name, COUNT(f.feedback_id) AS total_ratings, p.rating AS
avg_rating " +

"FROM product p LEFT JOIN feedback f ON p.product_id = f.product_id " + "WHERE

p.company_id = ? " +

"GROUP BY p.product_id, p.product_name, p.rating " +

"ORDER BY total_ratings DESC, avg_rating DESC";


try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

pstmt.setInt(1, companyId);

ResultSet rs = pstmt.executeQuery();


sb.append(String.format("%-30s %-15s %-10s\n", "Product Name", "Total Ratings", "Avg.
Rating"));

sb.append("-----\n");


boolean hasData = false;

while (rs.next()) {

hasData = true;

sb.append(String.format("%-30s %-15d %-10.2f ★\n",
rs.getString("product_name"),
rs.getInt("total_ratings"),
rs.getDouble("avg_rating")));

}

if (!hasData) {

sb.append("No products have been rated yet.");
}
}

```

```

    }
}

return sb.toString();
}

// SALES ANALYTICS
private void showSalesAnalytics() {
    try {
        String salesData = getSalesData();

        JTextArea textArea = new JTextArea(20, 70);
        textArea.setEditable(false);
        textArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
        textArea.setText(salesData);

        JScrollPane scrollPane = new JScrollPane(textArea);

        JOptionPane.showMessageDialog(null, scrollPane, "Sales Analytics (from Carts -
Simulation)", JOptionPane.INFORMATION_MESSAGE);

    } catch (Exception ex) {
        JOptionPane.showMessageDialog(null, "Error displaying sales analytics: " +
ex.getMessage());
    }
}

// GET SALES DATA FOR GUI
private String getSalesData() throws SQLException {

    StringBuilder sb = new StringBuilder();

    sb.append("=== SALES ANALYTICS (from User Carts - Simulation) ===\n\n");

    String sql = "SELECT p.product_name, " +
        "COALESCE(SUM(c.quantity), 0) as total_in_carts, " +

```

```
"COALESCE(SUM(c.quantity * p.price), 0) as total_revenue_in_carts " + "FROM  
product p " +  
"LEFT JOIN cart c ON p.product_id = c.product_id " +  
"WHERE p.company_id = ? " +  
"GROUP BY p.product_id, p.product_name " +  
"ORDER BY total_revenue_in_carts DESC";
```

```
try (Connection conn = Main.getConnection();
```

```
PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```
pstmt.setInt(1, currentCompanyId);
```

```
ResultSet rs = pstmt.executeQuery();
```

```
sb.append(String.format("%-25s %-15s %-20s\n", "Product Name", "Units in Cart",  
"Revenue in Carts"));
```

```
sb.append("-----\n");
```

```
double totalCompanyRevenue = 0;
```

```
int totalUnitsSold = 0;
```

```
boolean hasData = false;
```